

RUHR-UNIVERSITÄT BOCHUM

EINFÜHRUNG IN DIE PROGRAMMIERUNG

Arbeiten mit Feldern | Sommersemester 2026



Informatik
im Bauwesen

Einführung in die Programmierung – Arbeiten mit Feldern

Einführung

0	1	2	3	4	5	6	7	8	9
C	a	r	p	e	␣	D	i	e	m

0	1	2	3	4
55	0	33	0	0

0	1	2	3	4	5
true	true	false	true	true	true

- Viele Algorithmen beschäftigen sich mit der Verarbeitung von Daten
- Zusammenhängende Daten werden mit Hilfe Datenstrukturen organisiert
- Felder (List) sind die elementarsten Datenstrukturen
- Ein List speichert verschiedene Daten in zusammenhängenden Zellen
 - Jede Zelle enthält den gleichen Datentyp
 - Jede Zelle kann über einen Index (Ganzzahl) angesprochen werden
 - Die erste Zelle besitzt den Index 0

Einführung in die Programmierung – Einfache Datenstrukturen

List

- Ein in Python integriertes dynamisches Array
- Zugrundeliegende Implementierung: Basierend auf einem Array von PyObject-Zeigern (PyListObject)
- Mechanismus zur dynamischen Größenanpassung: Die Liste vergrößert oder verkleinert sich entsprechend, wenn Elemente eingefügt oder gelöscht werden.

Leistungsmerkmale:

- Indexzugriff
- Anhängen am Ende
- Einfügen/Löschen in der Mitte

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiele für einfache Felder

```
a = []  
b = [0, 0, 0]  
c = [0.0] * 100  
d = [False]
```

```
print ("length of a:", len(a))  
print ("length of b:", len(b))  
print ("length of c:", len(c))  
print ("length of d:", len(d))
```

- Python-Listen können in ihrer Größe verändert werden. Der `*` Operator kann zur schnellen Erstellung mehrerer gleicher Elemente verwendet werden
- In Python werden eckige Klammern `[]` verwendet, um Listen zu erstellen, die beliebige Datentypen enthalten können.

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiele für einfache Felder

```
length of a: 0  
length of b: 3  
length of c: 100  
length of d: 1
```

- Ein Liste kann die Länge (bzw. Größe) 0 besitzen
- Über das Attribut `len()` kann die Länge bestimmt werden
- Nach der Erzeugung kann die Länge nicht mehr geändert werden

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiele für einfache Felder

```
a.append(42)
a.append(7)
a.append(13)
print("a[0]:", a[0])
print("a[1]:", a[1])
print("a[2]:", a[2])
print("c[0]:", c[0])
print("d[0]:", d[0])
```

a[0]: 42
a[1]: 7
a[2]: 13
c[0]: 0.0
d[0]: False

- List-Elemente (Werte) werden mittels Index angesprochen
- Erster Index = 0 und letzter Index = n-1
- Beim Initialisieren einer Liste können die Listendaten nicht über den Index geändert werden, solange die Liste leer ist. Sie können die Funktion `append()` verwenden, um Daten zur Liste hinzuzufügen.

Einführung in die Programmierung – Arbeiten mit Feldern

Initialisierung

- Listen können bei der Initialisierung keine feste Länge haben. Die Länge lässt sich einfach festlegen, indem man der Liste wiederholt Werte zuweist. Anschließend werden den einzelnen Elementen mithilfe des Gleichheitsoperators `=` Werte zugewiesen.

```
b = [0.0] * 3  
b[0] = 1.0  
b[1] = -3.0  
b[2] = 9.5
```

- Alternativ können Sie den Elementen auch direkt bei der Initialisierung der Liste Werte zuweisen.

```
b = [1.0, -3.0, 9.5]
```

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiel: Summenbildung

Es soll eine Menge von n Zufallszahlen (zwischen 0 und 1) erzeugt werden, für welche anschließend die Summe gebildet werden soll

$$sum = \sum_{i=1}^n c_i$$

- Implementierung
- Liste mit n `float` Elementen erzeugen
- Jedem Element eine Zufallszahl mittels `random.random()` zuweisen
- Speicher für die Summe definieren
- Summe mittels `for` Schleife über alle Elemente bilden

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiel: Summenbildung

```
import random

c = [0.0] * 100
sum = 0.0
for i in range(len(c)):
    c[i] = random.random()
    sum += c[i]

print("sum of c:", sum)
```

Einführung in die Programmierung – Arbeiten mit Feldern

Fehler beim Zugriff

Gegeben sind die folgenden Liste

```
c = [0.0] * 100
```

Was passiert bei diesem Zugriff?

```
c[100] = 1
```

IndexError

```
c[100] = 1
    ~^^^^
```

```
IndexError: list assignment index out of range
```

Einführung in die Programmierung – Arbeiten mit Feldern

Fehler beim Zugriff

Gegeben sind die folgenden List

```
a = []
```

Was passiert bei diesem Zugriff?

```
a[0] = 42
```

IndexError

```
a[0] = 42
    ~^^^
```

```
IndexError: list assignment index out of range
```

Einführung in die Programmierung – Arbeiten mit Feldern

Fehler beim Zugriff

Fehlermeldungen genau lesen

```
a[0] = 42
    ~^^^
```

```
IndexError: list assignment index out of range
```

- Klasse, Methode und Zeilennummer werden angegeben
- Spezielle Informationen zum Fehler (hier welcher Index)

Einführung in die Programmierung – Arbeiten mit Feldern

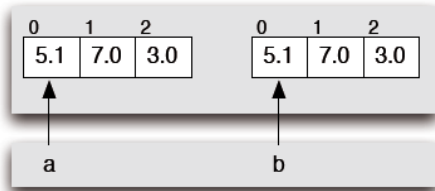
Liste sind Objekte

```
b = [5.1, 7.0, 3.1]
a = [5.1, 7.0, 3.1]
```

```
a = b
b[1] = 42.0
print("a[1]:", a[1])
```

```
a[1]: 42.0
```

Variablen für Liste speichern nur die Adresse des Liste



Einführung in die Programmierung – Arbeiten mit Feldern

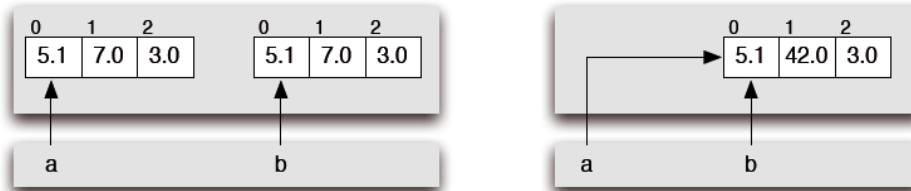
Liste sind Objekte

```
b = [5.1, 7.0, 3.1]
a = [5.1, 7.0, 3.1]
```

```
a = b
b[1] = 42.0
print("a[1]:", a[1])
```

```
a[1]: 42.0
```

Variablen für Liste speichern nur die Adresse des Liste



Einführung in die Programmierung – Arbeiten mit Feldern

Fehler beim Zugriff

```
rs = [None] * 2
r1 = Rectangle(0, 0, 1, 1)
print("rs[0]:", rs[0].is_inside(0, 0))
```

```
print("rs[0]:", rs[0].is_inside(0, 0))
                ^^^^^^^^^^^^^^^
```

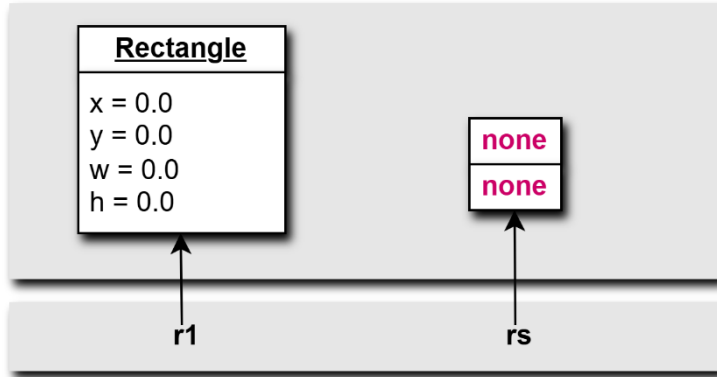
```
AttributeError: 'NoneType' object has no attribute 'is_inside'
```

Listen von Objekten

- In den Elementen werden nur Objektreferenzen (Speicheradressen) gespeichert
- Standardwert für die Elemente ist **None** (entspricht null-Zeiger)
- Ein **AttributeError** wird ausgelöst, wenn versucht wird, eine Methode auf **None** aufzurufen

Einführung in die Programmierung – Arbeiten mit Feldern

Fehler beim Zugriff



Listen von Objekten

- In den Elementen werden nur Objektreferenzen (Speicheradressen) gespeichert
- Standardwert für die Elemente ist **None** (entspricht null-Zeiger)
- Ein **AttributeError** wird ausgelöst, wenn versucht wird, eine Methode auf **None** aufzurufen

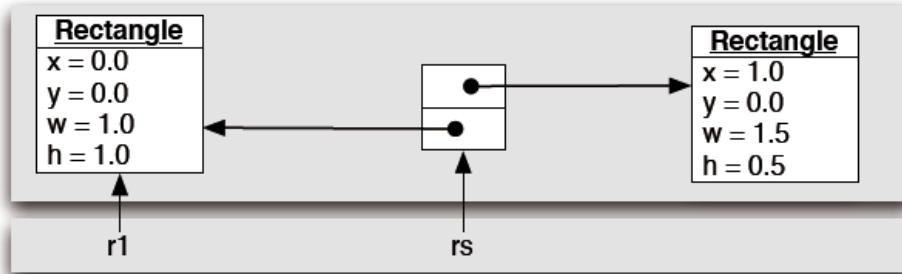
Einführung in die Programmierung – Arbeiten mit Feldern

Arbeiten mit Listen von Objekten

```
rs = [None] * 2  
r1 = Rectangle(0, 0, 1, 1)
```

```
rs[0] = r1  
rs[1] = Rectangle(1.0, 0.0, 0.5, 1.5)
```

```
print("rs[0]:", rs[0].is_inside(0, 0))  
print("rs[1]:", rs[1].is_inside(0, 0))
```



Einführung in die Programmierung – Arbeiten mit Feldern

Liste von Listen

Mehrdimensionale Liste

- Listen können beliebige Datentypen enthalten, auch gemischte Datentypen (1d Liste)

```
a = [0] * 999
```

- Ein List Datentyp kann sich wiederum aus einem List Datentyp zusammensetzen

```
b = [[0] * 2 for i in range(3)]
```

Liste von Listen von Ganzzahlen (2d Liste)

Einführung in die Programmierung – Arbeiten mit Feldern

Liste von Listen

Mehrdimensionale Liste

- Listen können beliebige Datentypen enthalten, auch gemischte Datentypen (1d Liste)

```
a = [0] * 999
```

- Ein List Datentyp kann sich wiederum aus einem List Datentyp zusammensetzen

```
c = [[ [0] * 3 for i in range(3) ] for i in range(3) ]
```

Liste von Listen von Listen von Ganzzahlen (3d Liste)

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiel: Mehrdimensionale Listen

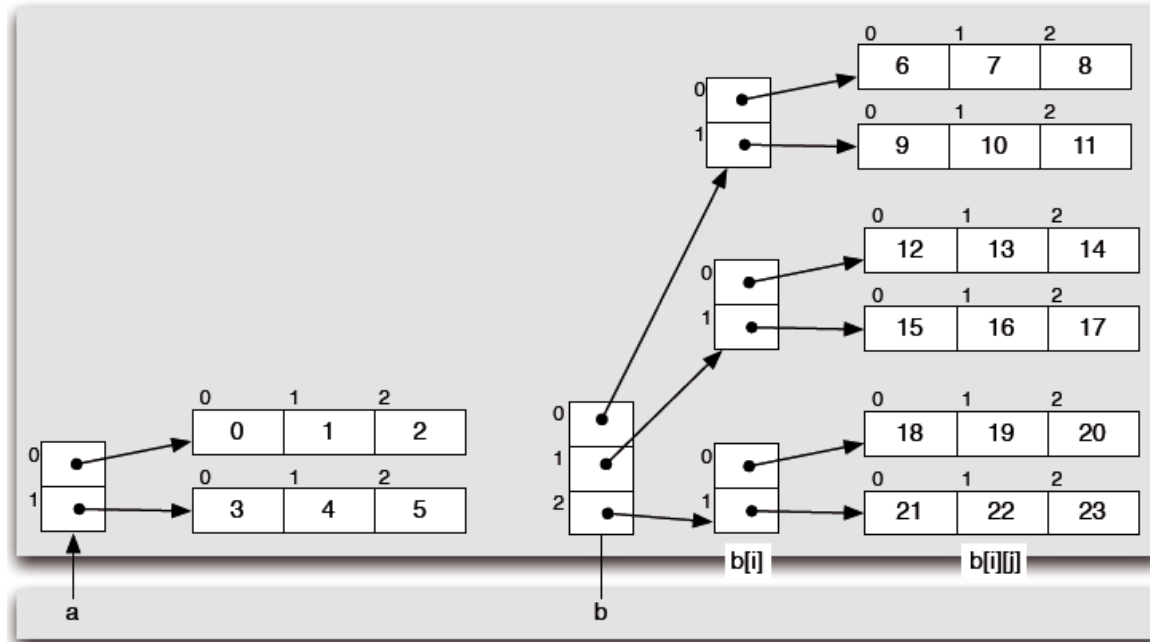
```
cnt = 0
a = [[0.0] * 2 for i in range(3)]
b = [[[0] * 3 for i in range(2)] for i in range(3)]
```

```
for i in range(len(a)):
    for j in range(len(a[i])):
        a[i][j] = float(cnt)
        cnt += 1

for i in range(len(b)):
    for j in range(len(b[i])):
        for k in range(len(b[i][j])):
            b[i][j][k] = cnt
            cnt += 1
```

Einführung in die Programmierung – Arbeiten mit Feldern

Beispiel: Mehrdimensionale Listen



Einführung in die Programmierung – Arbeiten mit Feldern

Beispiel: Arbeiten mit Vektoren

Vektoren werden in vielen Bereichen des Ingenieurwesens zur Beschreibung von Daten (latein. Träger) verwendet

- Geometrieinformationen
- Physikalische Werte

$$x = (x_1, x_2, \dots, x_n)^T$$

Entwicklung einer Klasse zum Arbeiten mit Vektoren

- Jeder Vektor ist ein eigenes Objekt
- Sinnvolle Methoden
 - Erzeugen eines Vektors
 - Abfrage und Manipulation der Vektorwerte
 - Ausgabe eines Vektors
 - Mathematische Operationen

Einführung in die Programmierung – Arbeiten mit Feldern

Mathematische Operationen

Skalarmultiplikation

$$\mathbf{y} = \alpha \mathbf{x} \text{ where } y_i = \alpha x_i$$

Allgemeine Vektoraddition

$$\mathbf{z} = \mathbf{x} + \alpha \mathbf{y} \text{ where } z_i = x_i + \alpha y_i$$

Skalarprodukt

$$(\mathbf{x}, \mathbf{y}) = \sum x_i y_i$$

Vektornormen

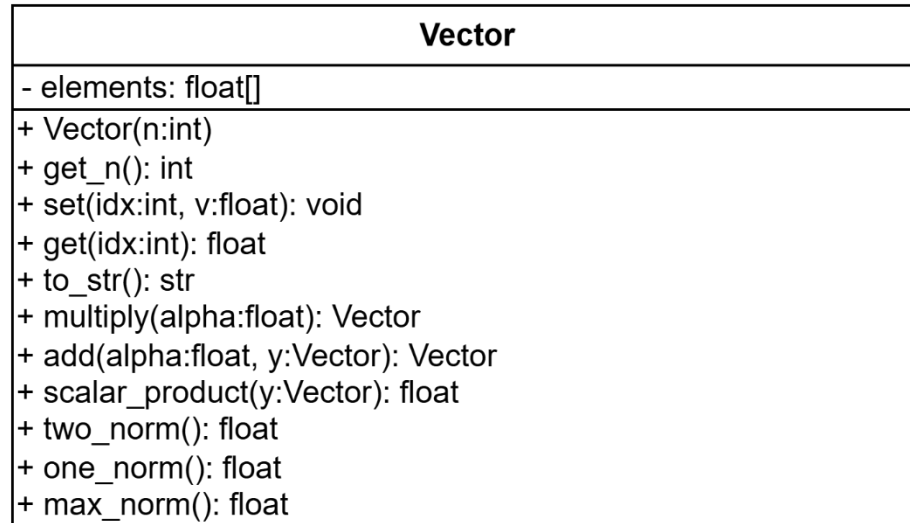
$$\|\mathbf{x}\|_2 = \sqrt{(\mathbf{x}, \mathbf{x})}$$

$$\|\mathbf{x}\|_1 = \sum |x_i|$$

$$\|\mathbf{x}\|_\infty = \max(x_1, x_2, \dots, x_n)$$

Einführung in die Programmierung – Arbeiten mit Feldern

UML Klassendiagramm



Vektorelemente werden als Liste von **float** Zahlen definiert

Einführung in die Programmierung – Arbeiten mit Feldern

Demoprogramm

```
x = Vector(4)
x.set(0, 2.0)
x.set(1, 3.0)
x.set(2, 4.0)
x.set(3, 5.0)

y = Vector(4)
y.set(0, 10.0)
y.set(2, 2.0)

z1 = x.multiply(2.0)
z2 = x.add(-1.0, y)

print("x:", x.to_str())
print("y:", y.to_str())
print("z1:", z1.to_str())
print("z2:", z2.to_str())
print("(x, y) =", x.scalar_product(y))
```

Einführung in die Programmierung – Arbeiten mit Feldern

Klasse für Vektoren – Attribute / Konstruktor

```
#Klasse für Vektoren
class Vector:
# Konstruktor
    def __init__(self, n):
        if n < 0:
            n = 0
        self.elements = [0.0] * n
```

Einführung in die Programmierung – Arbeiten mit Feldern

Klasse für Vektoren – Methoden

```
class Vector:
    # ...
    # Anzahl Vektorelemente ermitteln
    def get_n(self):
        return len(self.elements)

    # Wert v an der Stelle idx setzen
    def set(self, idx, v):
        if idx < 0 or idx >= len(self.elements):
            return float('nan')
        self.elements[idx] = v

    # Wert an der Stelle idx ermitteln
    def get(self, idx):
        # if Anweisung ... siehe oben
        return self.elements[idx]
```

Einführung in die Programmierung – Arbeiten mit Feldern

Klasse für Vektoren – Methoden

```
class Vector:
    # ...
    # Vektorwerte in eine Zeichenkette übernehmen
    def to_str(self):
        s = str(self.elements)
        return s
```

to_str Methode

- Wird durch `print` zur Ausgabe aufgerufen
- Ist eine Standardmethode aller Klassen und kann selber gestaltet werden (später mehr)

Einführung in die Programmierung – Arbeiten mit Feldern

Klasse für Vektoren – Methoden

```
class Vector:
    # ...
    # Vektorwerte in eine Zeichenkette übernehmen
    def to_str(self):
        s = str(self.elements)
        return s
```

Import einer anderen Klasse

- Durch eine **import** Anweisung wird der Pfad auf eine bestimmte Klasse festgelegt, die innerhalb der aktuellen Klasse verwendet werden soll

Einführung in die Programmierung – Arbeiten mit Feldern

Python Listen-Werkzeuge

- Die Klasse `List` enthält nützliche Methoden zum Arbeiten mit Listen
- Sortierung eines Listen
- Suche eines bestimmten Elements
- Prüft, ob zwei Listen die gleichen Elemente enthalten
- Gibt eine Liste in Form einer Zeichenkette zurück
- Kopiert einen bestimmten Teil eines Listen
-

Einführung in die Programmierung – Arbeiten mit Feldern

Geht es auch einfacher?

```
x = Vector(2.0, 3.0, 4.0, 5.0)
y = Vector(10.0, 0.0, 2.0, 0.0)
```

```
z1 = x.multiply(2.0)
z2 = x.add(-1.0, y)
```

```
print("x:", x.to_str())
print("y:", y.to_str())
print("z1:", z1.to_str())
print("z2:", z2.to_str())
print("(x,y) =", x.scalar_product(y))
```

Einführung in die Programmierung – Arbeiten mit Feldern

Variable Parameteranzahl

In Python wird durch die Angabe von `*n` eine variable Anzahl von Parametern ermöglicht

```
def __init__(self, *n):  
    if len(n) == 1:  
        if n[0] < 0:  
            l = 0  
        else:  
            l = n[0]  
        self.elements = [0.0] * l  
    else:  
        self.elements = list(n)
```

- Variable Parameterwerte werden automatisch in eine Liste vom entsprechenden Datentyp umgewandelt

Einführung in die Programmierung – Arbeiten mit Feldern

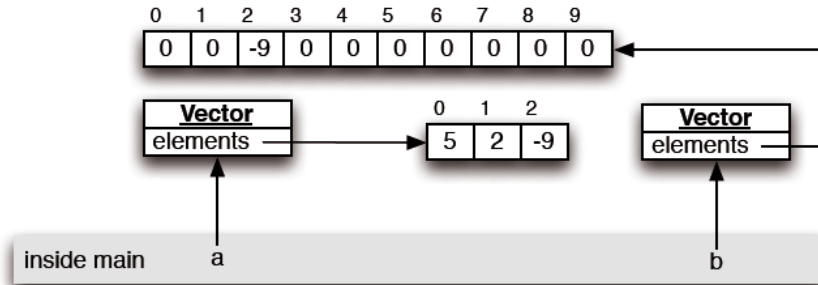
Zugriff auf Vektorelemente

- Auf die einzelnen Werte des Vektors kann mit Hilfe von `set` und `get` Methoden zugegriffen werden

```
a = Vector(5.0, 2.0, -9.0)
```

```
b = Vector(10)
```

```
b.set(2, a.get(2))
```



Einführung in die Programmierung – Arbeiten mit Feldern

Skalarmultiplikation

- Ergebnis soll als neuer Vektor zurückgegeben werden (**Vector**)
- Ausgangsvektor soll nicht verändert werden

```
class Vector:
    # Skalarmultiplikation
    def multiply(self, alpha):
        # Neuen Ergebnisvektor gleicher Länge erzeugen
        r = Vector(self.get_n())
        # Alle Elemente mit dem Skalar multiplizieren und
        # im Ergebnisvektor speichern
        for i in range(self.get_n()):
            r.set(i, alpha * self.get(i))
        return r
```

$$y_i = \alpha x_i$$

Einführung in die Programmierung – Arbeiten mit Feldern

Allgemeine Vektoraddition

- Ergebnis soll als neuer Vektor zurückgegeben werden (**Vector**)
- Ausgangsvektor soll nicht verändert werden

```
class Vector:
    # Vektoraddition
    def add(self, alpha, y):
        # Länge des Ergebnisvektor
        n = self.get_n()
        if self.get_n() > y.get_n():
            n = y.get_n()
        # Neuen Ergebnisvektor gleicher Länge erzeugen
        r = Vector(n)
        # Alle Elemente von y mit dem Skalar multiplizieren und
        # die Elemente von x addieren
        for i in range(n):
            r.set(i, self.get(i) + alpha * y.get(i))
        return r
```

$$z_i = x_i + \alpha y_i$$

Einführung in die Programmierung – Arbeiten mit Feldern

Skalarprodukt

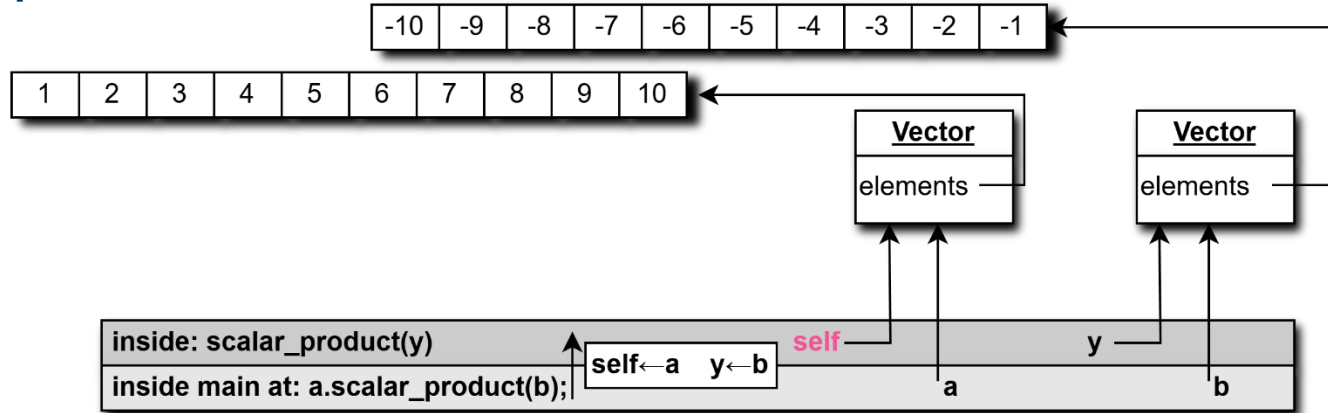
- Ergebnis ist eine reelle Zahl (`float`)
- Ergebnis muss am Anfang mit `0.0` initialisiert werden

```
class Vector:
    # Skalarprodukt
    def scalarProduct(self, y):
        # Länge des Ergebnisvektor
        n = self.get_n()
        if self.get_n() > y.get_n():
            n = y.get_n()
        # Speicher für das Skalarprodukt
        s = 0.0
        # Elemente von x mit den Elementen von y multiplizieren
        # und alles summieren
        for i in range(n):
            s += self.get(i) * y.get(i)
        return s
```

$$(x, y) = \sum x_i y_i$$

Einführung in die Programmierung – Arbeiten mit Feldern

Skalarprodukt



```
a = Vector(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
b = Vector(-10, -9, -8, -7, -6, -5, -4, -3, -2, -1)

s = a.scalar_product(b)
```

Einführung in die Programmierung – Arbeiten mit Feldern

Demoprogramm – Vektornormen

```
x = Vector(2.0, 3.0, 4.0, 5.0)
print("||x||2 =", x.two_norm())
print("||x||1 =", x.one_norm())
print("||x||i =", x.max_norm())
```

- Es wird nur die Implementierung der Euklidische Norm gezeigt
- Die anderen Normen werden in der Übung umgesetzt

Einführung in die Programmierung – Arbeiten mit Feldern

Euklidische Vektornorm

- Ergebnis ist eine reelle Zahl (`float`)
- Es wird die Wurzel des eigenen Skalarproduktes berechnet

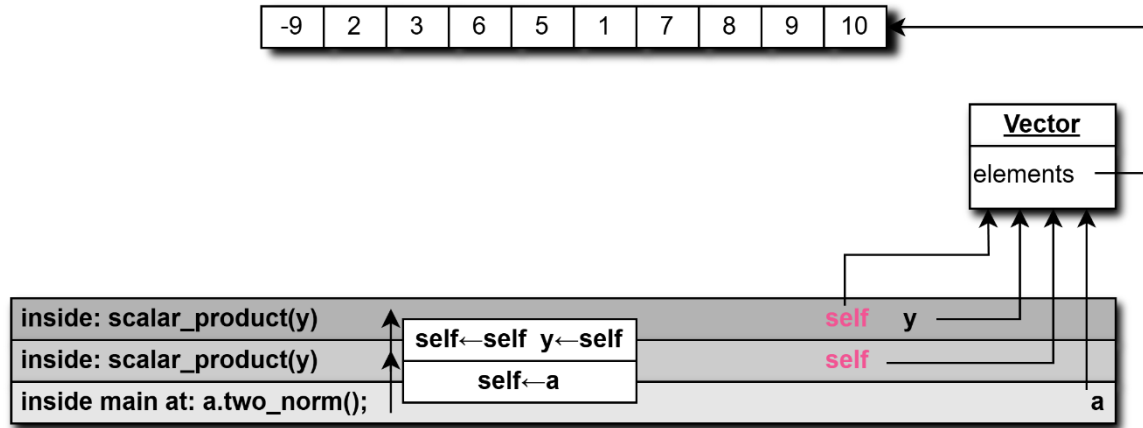
```
class Vector:
    # Euklidische Norm
    def two_norm(self):
        # Verwendung der Methode scalar_product
        return self.scalar_product(self) ** 0.5
```

$$\|x\|_2 = \sqrt{(x, x)}$$

Einführung in die Programmierung – Arbeiten mit Feldern

Euklidische Vektornorm

```
a = Vector(-9, 2, 3, 6, 5, 1, 7, 8, 9, 10)
s = a.two_norm()
```



- Beim Aufruf jeder einzelnen Methode werden die übergebenen Werte bzw. Speicheradressen in eigenen Parametervariablen zwischengespeichert